

T11 — Package, Moduli e Regole di Visibilità

Struttura Cartelle, Modificatori public/protected/default/private e MavenDate v2 con equals()

Docente: Prof. Michele Pasqua — **Appunti Revisione:** Wally

A.A. 2025/2026 – Università degli Studi di Verona

In questo blocco analizziamo la modularità avanzata offerta dai package, le regole di risoluzione dei nomi e i livelli di visibilità dei membri ([T11 - Packages and Class Visibility.pdf](#)). Sul piano pratico, in [Lezione11/MavenDate](#) assistiamo a una grande svolta architetturale: 1. Nascono le sottoclassi specializzate [ItalianDate](#) e [AmericanDate](#) tramite estensione (extends Date). 2. Viene introdotto il modificatore **protected**. 3. Viene implementato l'algoritmo canonico per l'overriding del metodo `equals(Object)`. 4. Viene risolto il bug storico dei getter `getMonth()` e `getYear()`.

0.0.1 1. Teoria: Concetti Teorici dalle Slide (Slide T11)

A. Modularità, Gerarchia e Convenzioni dei Package (Slide 1–11)

- **Definizione di Package:**

- Un package è un raggruppamento logico di classi correlate che scala il concetto di incapsulamento a livello di libreria.
- Definisce un **namespace isolato**: classi con lo stesso nome possono coesistere senza conflitti se collocate in package differenti (es. `it.oop.ui.Date` VS `it.oop.core.Date`).

- **Convenzione di Naming Standard:**

- Si usa il nome di dominio Internet invertito in caratteri minuscoli: `it.univr.mypackage`, `it.oop.core`.

- **Dichiarazione e Corrispondenza col Filesystem:**

- La direttiva `package nomepkg;` deve essere **la prima istruzione non di commento** all'inizio del file `.java`.
- Java impone una corrispondenza 1-a-1 tra la gerarchia dei package e la struttura delle cartelle su disco: la classe `it.oop.core.Date` deve trovarsi nel percorso `it/oop/core/Date.java`.

- **Sotto-Package e Wildcard:**

- I package possono essere annidati (es. `java.awt.event` è un sottopackage di `java.awt`).
- Attenzione: **Regola d'esame**: la direttiva `import pkg.*;` importa solo le classi direttamente contenute in `pkg`. **Non importa le classi contenute nei suoi sotto-package** (per importare `java.awt.event` è obbligatorio scrivere `import java.awt.event.*;`).

- **Il Default Package (Unnamed Package):**

- Se un file `.java` omette la dichiarazione `package`, la classe appartiene al package predefinito senza nome.
- **Limitazione critica:** Le classi nel default package **non possono essere importate né utilizzate** da classi situate in package con nome. Nel software professionale e agli esami il suo uso è fortemente scoraggiato.

- **Compilazione e Packaging con Strumenti Standard e Maven:**

- `javac -d bin/ src/pkg/MyClass.java`: il flag `-d` crea automaticamente l'albero delle sottocartelle nella directory di destinazione `bin/`.
- Con Maven: il layout `src/main/java` e `src/test/java` unito al `pom.xml` automatizza compilazione, test surefire e packaging in Fat JAR.

B. Visibilità di Classe e Matrice di Accesso dei Membri (Slide 12–19)

- **Visibilità a Livello di Classe (Top-Level Classes):**

- `public classClazz`: accessibile e istanziabile da qualsiasi package del Classpath.
- `classClazz` (*Package-Private* / default): visibile e utilizzabile **esclusivamente all'interno dello stesso package**. Non può essere usata all'esterno, anche se i suoi metodi interni sono dichiarati `public`.
- *Nota:* le classi di primo livello non possono essere dichiarate `private` né `protected` (questi modificatori sono ammessi solo per classi annidate/interne).

- **La Matrice dei Modificatori di Accesso (Fields e Methods):**

Modificatore	Stessa Classe	Stesso Package	Sottoclasse (Altro Package)	Mondo Esterno
private	[OK]	[NO]	[NO]	[NO]
Default (<i>package-private</i>)	[OK]	[OK]	[NO]	[NO]
protected	[OK]	[OK]	[OK]	[NO]
public	[OK]	[OK]	[OK]	[OK]

- **Risoluzione dei Nomi:**

- Se una classe importa due tipi omonimi o usa una classe locale avente lo stesso nome di una classe importata, per disambiguare è obbligatorio usare il **Fully Qualified Name (FQN)**: `it.oop.ui.Date`
`d = new it.oop.ui.Date(12345);`

0.0.2 2. Codice: Evoluzione del Codice: [Lezione11/MavenDate](#)

In `Lezione11`, il codice di `MavenDate` compie un balzo in avanti con l'introduzione di gerarchie di classi e l'overriding:

1. Generalizzazione in Date.java

```

1 package it.oop.core;
2
3 public class Date {
4     // 1. Campo 'protected': accessibile direttamente dalle sottoclassi
5     protected final int day;
6     // 2. Campi 'private': incapsulati e accessibili solo tramite getter
7     private final int month;
8     private final int year;
9
10    public Date(int day, int month, int year) {
11        this.day = day;
12        this.month = month;
13        this.year = year;
14        verify();
15    }
16
17    // 3. I GETTER SONO STATI CORRETTI!
18    public int getDay() { return day; }
19    public int getMonth() { return month; } // Non restituisce più day!
20    public int getYear() { return year; } // Non restituisce più day!
21
22    // 4. Implementazione canonica di equals(Object)
23    @Override
24    public boolean equals(Object other) {
25        if (other == null) return false;           // Controllo null-safety
26        if (this == other) return true;           // Controllo identità (stesso indirizzo)
27        if (!(other instanceof Date)) return false; // Controllo di tipo/ereditarietà
28        Date otherAsDate = (Date) other;          // Downcast sicuro
29        return day == otherAsDate.getDay()         // Confronto dei campi
30            && month == otherAsDate.getMonth()
31            && year == otherAsDate.getYear();
32    }
33 }

```

Analisi: I 5 Passi Fondamentali dell'Algoritmo equals(Object):

1. `if (other == null) return false;`: garantisce che `d.equals(null)` ritorni `false` senza lanciare `NullPointerException`.
2. `if (this == other) return true;`: ottimizzazione immediata (riflessività). Se sono lo stesso oggetto nello Heap, sono identici.
3. `if (!(other instanceof Date)) return false;`: verifica se l'oggetto passato è compatibile con la gerarchia `Date`.
4. `Date otherAsDate = (Date) other;`: downcast necessario perché la firma di `equals` accetta un generico `Object`.
5. Confronto logico dei campi che definiscono lo stato: `day`, `month`, `year`.

2. Le Nuove Sottoclassi: ItalianDate.java e AmericanDate.java

```

1 package it.oop.core;
2
3 public class ItalianDate extends Date {
4     private static final String FORMAT = "dd/mm/yyyy";
5     private static final String[] MONTHS = { "gennaio", "febbraio", ... };
6
7     // Invocazione del costruttore genitore con 'super'
8     public ItalianDate(int day, int month, int year) {
9         super(day, month, year);
10    }
11
12    public String printFormat() { return FORMAT; }

```

```

13
14     public String prettyPrint() {
15         // 'day' è accessibile direttamente perché 'protected'!
16         // 'getYear()' è invocato tramite metodo perché 'year' è 'private' in Date!
17         return day + " " + getMonthAsString() + " " + getYear();
18     }
19
20     @Override
21     public String toString() {
22         return day + "/" + getMonth() + "/" + getYear();
23     }
24 }

```

E simmetricamente per `AmericanDate`:

```

1 public class AmericanDate extends Date {
2     private static final String FORMAT = "mm/dd/yyyy";
3
4     public AmericanDate(int day, int month, int year) {
5         super(day, month, year);
6     }
7
8     @Override
9     public String toString() {
10        // Mese prima del giorno!
11        return getMonth() + "/" + getDay() + "/" + getYear();
12    }
13 }

```

3. Polimorfismo, Upcasting e Downcasting in `MainDate.java` Analizziamo i comportamenti a runtime testati dal docente:

```

1 Date date = new Date(3, 11, 2025);
2 ItalianDate itDate = new ItalianDate(3, 11, 2025);
3 AmericanDate usDate = new AmericanDate(3, 11, 2025);
4
5 // 1. Confronti con equals():
6 System.out.println("date ?= itDate: " + date.equals(itDate)); // Stampa TRUE!
7 System.out.println("itDate ?= usDate: " + itDate.equals(usDate)); // Stampa TRUE!

```

Perché stampano `true`? Perché sia `itDate` che `usDate` estendono `Date`: l'operatore `instanceof Date` è soddisfatto, e i campi (giorno 3, mese 11, anno 2025) coincidono perfettamente!

```

1 // 2. Upcasting (Polimorfismo di Inclusione):
2 Date d = new ItalianDate(1, 1, 1970); // LECITO: ItalianDate È UN Date!
3
4 // 3. Tipi Incompatibili tra rami fratelli:
5 // AmericanDate ad = new ItalianDate(1, 1, 1970); // COMPILE-TIME ERROR!
6
7 // 4. Downcasting esplicito lecito:
8 ItalianDate id = (ItalianDate) d; // LECITO a runtime: l'oggetto nello Heap è davvero ItalianDate!
9 System.out.println(id.printFormat()); // Stampa dd/mm/yyyy
10
11 // 5. Downcasting illegale a runtime:
12 // ItalianDate idFalso = (ItalianDate) date; // CRASH RUNTIME: ClassCastException!
13 // (perché 'date' è un Date generico, non contiene la specializzazione ItalianDate)

```

0.0.3 3. Ciclo: Corrispondenza con i Tuoi Moduli Workspace

Nel tuo repository [esercizi-wally-25-26](#): - **es13**: [Calculator.java](#) e [TestCalculator.java](#) applicano la separazione `es13.model` e `es13` illustrata a lezione (Slide T11, pp. 3–14). - **es14**: implementa esattamente questa architettura a oggetti multi-package (`it.oop.core` e `it.oop.ui`) con [ItalianDate](#), [AmericanDate](#), [Date](#) e i relativi test in [src/test/java/it/oop/core/TestItalianDate.java](#).

Nota di Studio

Con la **Lezione 11** abbiamo visto l'introduzione dell'ereditarietà (`extends`), la visibilità `protected`, l'overriding di `equals()` e le regole di cast tra classi.

Il prossimo blocco è la **Lezione 12 / Slide T12: *Inheritance and Polymorphism***: - Teoria approfondita sull'ereditarietà: riuso, estensione e specializzazione. - La classe radice universale `java.lang.Object` e i suoi metodi (`toString`, `equals`, `hashCode`, `getClass`, `clone`). - Il costrutto `super`: invocazione di costruttori (`super(...)`) e invocazione di metodi della superclasse (`super.metodo()`). - **Polimorfismo e Dynamic Method Dispatch (Late Binding)**: come la JVM decide a runtime quale metodo eseguire tramite la `vtable`. - Regole di overriding vs overloading e l'annotazione `@Override`. - Evoluzione del codice in Lezione12: - La classe `Date` diventa la superclasse polimorfica di riferimento. - Aggiunta del metodo `isLeapYear(int year)` per la validazione completa dei bisestili gregoriani!

Dammi conferma per aprire la Lezione 12 / T12 e proseguire!